

DAY 6

Task1

- Simple Lambda function to greet Good morning

```
interface Greeting{
    void saymessage();
}
public class Day6_LambdaGreeting {
    public static void main(String[] args) {
        Greeting greeting = () -> System.out.println("Good morning");
        greeting.saymessage();
    }
}
```

Task 2

```
package org.example; in correct
interface Prod{
    int multiply(int a, int b);
}
public class Demo034_LambdaExp w{
    public static void main(String[] args) {
        Prod multiply = (a, b) -> a * b;

        int result = Prod.multiply(10, 20);
        System.out.println("prod is : " + result);
    }
}
```

Corrected code

```
package org.example;
interface Math {
    int compute(int a, int b);
}
public class Demo034_LambdaExp {
    public static void main(String[] args) {
        Math multiply = (a, b) -> a * b;
        int result = multiply.compute(10, 20);
        System.out.println("prod is : " + result);
    }
}
```

Task 3

Wap to create a calc using lambda exp

```
interface Calculatorr {
    int compute(int a, int b);
}

public class Day6_LambdaCalculator {
    public static void main(String[] args) {
        Calculatorr add = (a,b) -> a + b ;
        Calculatorr sub = (a,b) -> a - b ;
        Calculatorr Multiply = (a,b) -> a * b ;

        System.out.println("Addition: " + add.compute(100, 50));
        System.out.println("Subtraction: " + sub.compute(100, 50));
        System.out.println("multiply: " + Multiply.compute(100, 50));
    }
}
```

Task 4

You need to create an interface

-> With one abstract method (SAM) and 1 normal method - Yes

→ can you create 2 normal methods ? -Yes

→ can i have a method with abstract keyword -yes

```
@FuncInterface
interface FuncInterface {
    abstract void abstractFun(int x);
    // abstract void abstractFun2(int x); //2nd abstract
    // void abstractFun1(int x); // abstract not rec
    default void normalFun() {
        System.out.println("Welcome");
    }
}

public class Demo035_LambdaExp2 {
    public static void main(String[] args) {
        FuncInterface fobj = (int val) -> System.out.println(20 * val);
        fobj.abstractFun(50);
        fobj.normalFun();
        // fobj.abstractFun1();
    }
}
```

```
}  
}
```

Task 5

What is this code snippet doing..

Can i have a zero parameter lambda exp or not?

```
package org.example;  
@FunctionalInterface  
interface OParameter {  
    void display();  
}  
public class Demo036_LambdaExp3 {  
    public static void main(String[] args) {  
        OParameter obj = ()  
            -> System.out.println(  
                "This is a zero-parameter lambda expression!");  
        obj.display();  
    }  
}
```

- Yes , we can have Zero parameter lambda expression.

Task 6

Wap to create an array list

Use lambda exp in foreach

Check if the num is even or odd

```
import java.util.ArrayList;  
  
public class Day6_LambdaArray {  
    public static void main(String[] args) {  
        ArrayList<Integer> numbers = new ArrayList<>();  
        numbers.add(10);  
        numbers.add(15);  
        numbers.add(22);  
        numbers.add(37);  
        numbers.add(40);  
        numbers.forEach(num -> {
```

```

        if (num % 2 == 0) {
            System.out.println(num + " is Even");
        } else {
            System.out.println(num + " is Odd");
        }
    });
}
}

```

-Assertion

```

public class Day6_Test {
    public static void main(String[] args) {
        int age = 20;
        assert age >= 18: "can vote";
        System.out.println("Your " + age);
    }
}

```

Task 7

What is Junit 5?

- Junit 5 is latest version of Junit testing Framework used to run unit tests.
- It can only be used in Java
- It helps user to verify that the individual methods and classes work correctly by executing the test cases automatically.

Task 8

List down the annotations of Junit 5

1. `@Test` – marks the method as a test method
2. `@BeforeEach` -Execute before each test method
3. `@AfterEach` - Execute after each test method
4. `@AfterAll` – Executes once after all test methods
5. `@BeforeEach` - Executes once before all test methods

Task 9

Difference between Junit 5 and Junit 4

- JUnit 4 is an older version of the JUnit testing framework that uses annotations such as `@Before`, `@After`, `@BeforeClass`, and `@AfterClass` for test execution.
- JUnit 5 introduces a modular architecture with new annotations like `@BeforeEach`, `@AfterEach`, `@BeforeAll`, and `@AfterAll`, along with improved support for Java 8 features, parameterized tests, and extensions.
- JUnit 5 is more flexible, powerful, and easier to extend than JUnit 4.

Task 10

What is JUnit Test Suite?

- It is a collection of multiple test classes or test cases that are grouped together and executed as a single unit. It allows developers to run several related tests at once instead of executing each test class individually.
- Test suites are useful for organizing tests, simplifying test execution, and ensuring that all related functionality is tested together.

Task 11

What is JUnit Vintage?

- It is a test engine provided in JUnit 5 that allows developers to run JUnit 3 and JUnit 4 test cases on the JUnit 5 platform.
- It provides backward compatibility, enabling older tests to work with the new JUnit 5 framework without rewriting them.
- This helps in gradually migrating projects from JUnit 4 to JUnit 5.

Task 12

What are mock objects?

- Mock objects are simulated objects used in unit testing to imitate the behavior of real objects or dependencies
- Mock objects make tests faster, more reliable, and easier to control.

Task 13

Write about Mockito?

- It is Java framework used for creating and managing mock objects in unit testing.
- It allows developers to test classes independently by mocking external dependencies and defining their expected behavior.
- Mockito helps in writing simple and reliable unit tests by verifying method calls, returning predefined values, and simulating different scenarios without using real objects.

Task 14

Define Functional Interface?

- It is an interface in Java that contains exactly one abstract method.
- It can have multiple default and static methods, but only one abstract method is allowed.
- Functional interfaces are mainly used with lambda expressions and method references in Java 8.

Task 15

What are default and static methods in interfaces?

- A default method uses the `default` keyword and can be inherited and overridden by implementing classes
- A static method belongs to the interface itself and is called using the interface name. These methods allow interfaces to add new functionality without breaking existing implementations.

Task 16

In Java 8 stream API

- Java 8 Stream API is a feature that allows developers to process collections of data in a simple and efficient way using functional programming concepts.
- It provides operations like filtering, mapping, sorting, and reducing data without modifying the original collection. Stream API improves code readability and supports parallel processing for better performance when working with large amounts of data.

```
import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

public class Day6_StreamAPI {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>();

        list.add("Monika");
        list.add("Likitha");
        list.add("Thanu");
        list.add("Likhi");

        List<String> filtered = list.stream()
            .filter(name -> name.startsWith("L"))
            .collect(Collectors.toList());

        System.out.println(filtered);
    }
}
```

Task 17

What is Optional class in Java 8?

- It is a container object used to represent a value that may or may not be present.
- It helps avoid NullPointerException .
- Optional provides methods like `isPresent()`, `orElse()`, and `ifPresent()` to check, retrieve, and handle values effectively.

Task 18

What is error in the below Optional class – which is a feature of java 8?

```
package org.example;

import java.util.Optional;

public class Demo040_Optional {
    public static void main(String[] args) {
        String user = "Prasunamba";
        Optional.ofNullable(user).map(User::getName)
    ).orElse("Guest"); [1];
    }
}
```

-

Task 19

What is the double colon :: operator ?

- It is called the method reference operator. It is used to refer to an existing method or constructor without executing it. It is mainly used with functional interfaces and lambda expressions to make code shorter and more readable.

Task 20

What is method Reference in java 8?

- A method reference in Java 8 is a shorthand way of writing a lambda expression by referring to an existing method or constructor using the double colon (::) operator as said in Task 19.

Task 21

There are different types of method reference

· Types:

- o Static methods (Math::max)
- o Instance methods (String::toLowerCase)
- o Constructors (ArrayList::new)

Wap to use the above types of methods and show the use of each type

```
import java.util.ArrayList;
import java.util.function.BiFunction;
import java.util.function.Function;
```

```

import java.util.function.Supplier;

public class Day6_MethodReference{

    public static void main(String[] args) {
        BiFunction<Integer, Integer, Integer> max = Math::max;
        int result = max.apply(10, 20);
        System.out.println("Maximum value: " + result);
        Function<String, String> lowerCase = String::toLowerCase;
        String name = lowerCase.apply("JAVA METHOD REFERENCE");
        System.out.println("Lowercase String: " + name);

        Supplier<ArrayList<String>> listCreator = ArrayList::new;

        ArrayList<String> list = listCreator.get();

        list.add("Java");
        list.add("Python");
        list.add("C++");

        System.out.println("ArrayList values: " + list);
    }
}

```

Task 22

Java 8 one more feature is date and time API

Modern date and time

localDate(date) ,

localTime(time),

localDateTime(both)

ZoneDateTime(timezone)

Don't use legacy..

Java.util.Date;

java.util.Calendar;

```

import java.time.LocalDate;
import java.time.LocalTime;
import java.time.LocalDateTime;
import java.time.ZonedDateTime;
import java.time.ZoneId;

public class Day6_DataTime {

    public static void main(String[] args) {

```

```
LocalDate date = LocalDate.now();
System.out.println("Current Date : " + date);

LocalTime time = LocalTime.now();
System.out.println("Current Time : " + time);

LocalDateTime dateTime = LocalDateTime.now();
System.out.println("Current DateTime : " + dateTime);

ZonedDateTime zonedDateTime =
    ZonedDateTime.now(ZoneId.of("Asia/Kolkata"));
System.out.println("Current ZonedDateTime : " + zonedDateTime);

}
}
```